SDN Orchestration of Multi-Domain QKD Networks: the ETSI GS QKD 018 Interface

Diego Carro and Bucci Felice Elia

Cybersecurity course, Politecnico di Torino
Quantum Engineering

November 2026

Abstract

Quantum key distribution (QKD) lets two parties agree on a secret key whose security rests on the laws of physics rather than on computational hardness. Moving it from the laboratory into an operator's network, however, means managing it like any other network resource: discovering its topology, monitoring its state, allocating its capacity to services and coordinating it with classical networks. The ETSI ISG QKD addressed this problem step by step. GS QKD 002 identifies the use cases and integration requirements; GS QKD 015 defines the model through which an SDN controller sees and configures the QKD nodes of a domain; GS QKD 018 defines the interface between an *SDN orchestrator* and the controller of a QKD domain, enabling end-to-end provisioning across several domains and between a QKD network and an optical transport network. This work follows that chain in a reasoned way: it describes the components of an SD-QKD network and the structure of a YANG model, analyses GS QKD 018 in detail (use cases, information model, topology, inventory and connectivity modules, multi-domain workflow, security), shows its analogy with the IETF ACTN framework (RFC 8453) and compares it with three real experiences: SK Telecom's multi-vendor KOREN testbed, an ONOS-based hierarchical prototype developed at Politecnico di Milano, and the MadQCI network in Madrid. It closes with the current limits of the standard and open research directions.

Keywords: QKD, SDN, orchestration, ETSI GS QKD 015, ETSI GS QKD 018, YANG, RESTCONF, multi-domain, key management.

Contents

1 State of the art **2**

2 Introduction **3**

2.1 ETSI GS QKD 002: the use cases 5

2.1.1 QKD as a cryptographic primitive 5

2.1.2 Where QKD fits in the network stack 5

2.1.3 The six use cases 6

2.1.4 What it leaves behind 7

2.2 ETSI GS QKD 015: the SDN controller and the SD-QKD network 7

2.2.1 Why software-defined control 8

2.2.2 The components of an SD-QKD network 8

2.2.3 Anatomy of a YANG model 10

2.2.4 The SD-QKD node model 12
2.2.5 Notifications 14
2.2.6 Workflows 14
2.2.7 Protocols and security 15
2.2.8 From a single domain to orchestration 15

3 ETSI GS QKD 018 16
3.1 GS QKD 018 use cases 16
3.2 The information model 17
3.3 The YANG modules 18
3.4 Multi-domain provisioning workflow 20
3.5 Protocols and security 21

4 The analogy with ACTN (RFC 8453) 21

5 Real deployments 22
5.1 SK Telecom: multi-vendor orchestration on KOREN 22
5.2 Politecnico di Milano: hierarchical control with ONOS 22
5.3 MadQCI: the Madrid network 22

6 Limits and outlook 22

7 Conclusions 23

References 23

1 State of the art

QKD fundamentals. The QKD literature is by now well established. The review by Pirandola et al. [1] covers discrete-variable protocols (BB84 and its *decoy-state* variants), continuous-variable protocols and security proofs. Two constraints that drive all network engineering emerge from it: the key rate decays exponentially with distance, and a single link over commercial fibre does not exceed a few hundred kilometres.

From link to network. Mehic et al. [2] and Cao et al. [3] describe the transition from point-to-point links to QKD *networks*: layered architecture, *trusted relay*, key management and routing. The GÉANT white paper [4] gives a readable summary of the same picture and of the ETSI and ITU-T standards. Routing, which becomes central once a network is orchestrated, is discussed in [5].

Standardisation. Two bodies have produced complementary specifications. ETSI ISG QKD started in 2010 with the use cases [6] and then defined the interfaces towards applications [7, 8], towards the SDN controller [9], towards the orchestrator [10] and between key management systems [11]. ITU-T, with the Y.3800 series, defined the reference architecture of QKD networks (QKDN) [12], their control and management [13] and SDN-based control [14].

SDN applied to QKD. The *Software-Defined Networking* paradigm [15] separates the control plane from the data plane and centralises decisions in a programmable controller. The UPM/Telefónica group was the first to integrate QKD resources into an SDN-controlled optical network, both experimentally [16] and in a systematic tutorial [17]; GS QKD 015 grew out of this line of work.

Orchestration and multi-domain. The orchestrator–controller interface is specified by GS QKD 018 [10] and its YANG modules [18]. The first documented real deployment is that of Sim et al. [19] (SK Telecom), who orchestrate two domains from different vendors on the KOREN testbed. Vicario's thesis [20] builds a complete hierarchical prototype with ONOS and RESTCONF. The MadQCI network [21] uses GS QKD 015 extensively but describes GS QKD 018 orchestration as work still in progress. More recent works propose SDN applications for dynamic link reconfiguration [22], path-selection criteria and inter-domain routing that preserves topology confidentiality [23], and hybrid key-delivery models between regional networks [24].

Contribution. With respect to these sources, this work offers a unified and reasoned reading of the GS QKD 002 $\rightarrow$ 015 $\rightarrow$ 018 chain: it shows how the requirements of the use cases translate into the components of an SD-QKD network, explains the structure of the YANG models that describe them and analyses how GS QKD 018 abstracts them for multi-domain orchestration.

2 Introduction

This section builds the vocabulary needed to read the ETSI specifications. It starts from QKD as a cryptographic primitive, moves on to QKD networks with their trusted nodes and layered architecture, recalls the essential concepts of SDN and finally reaches the two documents that precede GS QKD 018: GS QKD 002 (§2.1) and GS QKD 015 (§2.2).

Quantum key distribution.

Definition 2.1 (QKD)**.** *Quantum Key Distribution* is a procedure that, through the transport of quantum states, allows two remote parties to agree on shared secret bit strings, with security based on entanglement or on the impossibility of perfectly cloning or measuring unknown quantum states [10].

BB84 makes the idea clear. Alice encodes random bits on single photons in one of two randomly chosen bases; Bob measures in a basis that is also random; over an *authenticated* classical channel the two compare bases and keep only the bits measured in the right one (*sifting*). An eavesdropper who measures the photons unavoidably introduces errors, quantified by the *Quantum Bit Error Rate*

$$\mathrm{QBER} = \frac{N_{\mathrm{errors}}}{N_{\mathrm{sifted}}}. \tag{1}$$

If the QBER stays below a threshold (about 11% for BB84 against collective attacks), error correction and *privacy amplification* distil a secret key; otherwise the block is discarded. The quantity the network cares about is the *secret key rate* (SKR), the number of secure key bits produced per second. Since fibre transmittance decays as $\eta = 10^{-\alpha L/10}$, with $\alpha \approx 0.2$ dB/km, the SKR also drops quickly with the link length L.

Two observations, which we will meet again in the GS QKD 002 use cases, are fundamental. QKD *distributes* keys but neither encrypts nor authenticates: it must always be combined with a cipher (one-time pad or AES) and an authentication mechanism. Moreover, the classical channel

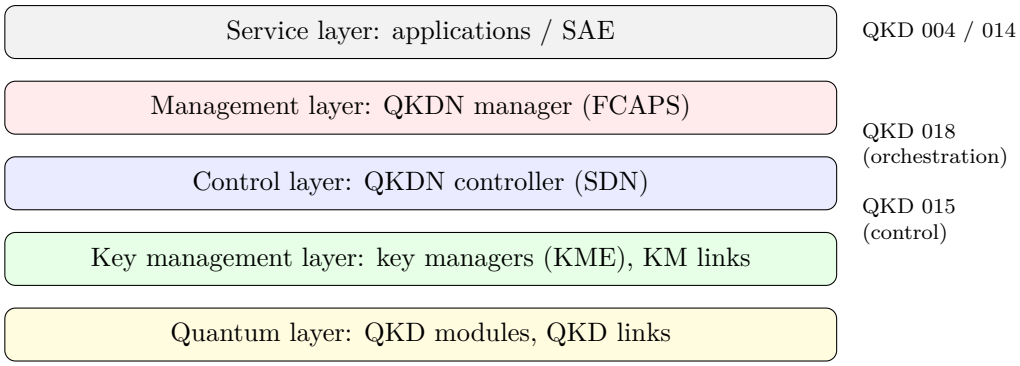


Figure 1: Layered architecture of a QKD network according to ITU-T Y.3800 [12]; on the right, the indicative position of the ETSI specifications discussed in the text.

must be authenticated from the very first exchange, which requires a small pre-shared secret: strictly speaking QKD is a *key growing* mechanism.

From link to network: the trusted node. A QKD link spans at most a few hundred kilometres and practical quantum repeaters do not exist yet. Real networks therefore chain links through trusted nodes.

Definition 2.2 (Trusted node and key relay)**.** A *trusted node* is a node, located inside a physically protected perimeter, that shares QKD keys with both its neighbours. To carry a key K from A to C through B, node A sends $c = K \oplus K_{AB}$; B computes $K = c \oplus K_{AB}$ and forwards $c' = K \oplus K_{BC}$. Each hop is *one-time pad* encrypted, but K appears in the clear inside every intermediate node, which must therefore be trusted.

Note that relaying consumes key: to carry $|K|$ bits along a path p, every link $\ell \in p$ spends $|K|$ bits of its own local key. This simple fact turns key into a *network resource* that must be planned, and it is why the later specifications talk about available rates, consumption and paths.

The ITU-T layered architecture. Recommendation ITU-T Y.3800 [12] organises a QKD network (QKDN) into stacked layers (Fig. 1): the *quantum layer* with the QKD modules; the *key management layer* with the key managers that store and relay keys; the *control layer*, which takes operational decisions (routing, relay, link configuration); the *management layer*, which supervises everything; and the *service layer*, where applications consume keys. Two kinds of link are distinguished in this scheme.

Definition 2.3 (QKD link and KM link)**.** A *QKD link* connects two QKD modules and consists of a quantum channel and a classical channel: it is where key is *generated*. A *KM link* connects two *key managers*: it is the classical channel, protected with QKD keys, along which key is *relayed*.

Likewise, two roles that are often confused are kept apart. The *QKDN controller* acts on the network in real time: it decides relay paths, enables or disables links and reacts to failures. The *QKDN manager* has a supervisory role: it collects FCAPS information (*Fault, Configuration, Accounting, Performance, Security*) and enforces policies. The ETSI specifications discussed below sit in the control plane: GS QKD 015 between controller and nodes, GS QKD 018 between controller and orchestrator.

Applications do not see the quantum part. A *Secure Application Entity* (SAE) requests keys from the *Key Management Entity* (KME) of its node through the GS QKD 004 [7] or GS QKD 014 [8] APIs; two SAEs that want to communicate obtain the same key from their respective KMEs, identified by a *key ID*.

Software-Defined Networking. In a traditional network each device decides on its own how to forward traffic. SDN [15] instead separates three planes: the *data plane*, made of devices that execute; the *control plane*, centralised in a controller with a global view of the network; and the *management and application plane*, which expresses goals. The controller talks to the devices through a *southbound* interface (SBI) and offers its services upwards through a *northbound* interface (NBI). When there is more than one domain, an *orchestrator* is added above the controllers to coordinate end-to-end services without knowing the details of each domain.

A QKD network has every feature that makes SDN useful: scarce and variable resources (the key), tight physical constraints (attenuation, coexistence with classical channels), multi-vendor equipment and demand that changes over time. The two specifications that follow describe why and how ETSI applied this paradigm to QKD.

2.1 ETSI GS QKD 002: the use cases

GS QKD 002 V1.1.1 (June 2010) [6] is the first document published by the ISG QKD. Its goal is not strictly technical: before standardising *how* to implement QKD, the group wanted to clarify *where* it is needed, *who* uses it and *which requirements* it must meet to enter existing infrastructures. The document reasons in three steps: it places QKD among cryptographic primitives, identifies the layers of the network stack where it can be inserted and describes six use cases with their requirements. Following this reasoning is useful because it shows where the needs come from that, twelve years later, led to GS QKD 015 and 018.

2.1.1 QKD as a cryptographic primitive

The document starts from a typical QKD system: two units, usually 19" rack-mounted, connected by a quantum channel over fibre or free space up to about 100 km long, which output the same random bit sequence at both ends. Clause 4 immediately makes clear that this solves *a single* cryptographic problem.

A secure system needs three primitives: *encryption*, *key distribution* and *message authentication*. QKD covers only the second; encryption requires a one-time pad or symmetric ciphers such as AES, and authentication requires MACs (for instance based on Wegman–Carter universal hashing) or digital signatures. It follows that QKD is a *complement* to classical cryptography, not a replacement, and that its overall security also depends on the other components of the system. The document also stresses that the public channel of QKD must be authenticated with a pre-distributed secret.

2.1.2 Where QKD fits in the network stack

Clause 6 describes at which layer of the stack a QKD key can be consumed. The choice of layer determines what must be modified and how transparent the integration is.

Data link layer (L2)
The QKD key feeds the PPP key exchange, the IEEE 802.1AE (MACsec) keys or a *QKD link encryptor*, i.e. a line encryptor that directly consumes the keys produced by the link and creates an encrypted tunnel between two sites. It is the most transparent option for applications.

Network layer (L3)
A modified IKE uses the QKD key as the shared secret for IPsec.

Transport layer (L4)
The QKD key replaces the TLS/SSL session keys or is used instead of HMACs for authenti-

cation.

Application layer
Applications use the key to authenticate users or to encrypt the content exchanged between their instances.

The conceptual point is that, whatever the layer, there is one component that *consumes* keys and one that *supplies* them: this is the separation that later becomes SAE and KME, with a standard interface (GS QKD 004 and 014) between them.

2.1.3 The six use cases

Clause 7 describes six use cases with a fixed structure: goal, description, actors, benefits, operational and quality-of-service considerations. We summarise them along a logical thread: from a single link to a network, and from a private network to an operator's network.

UC1 – Offsite backup / business continuity. An enterprise replicates data from a primary site to a backup site (daily backups, snapshots, SAN mirroring, database replication). A single point-to-point QKD link feeds a line encryptor over Ethernet or Fibre Channel; the fibre may be dedicated (*dark fibre*) or shared via WDM or TDM. The requirements that emerge are: key renewal as frequent as the generation rate allows, handling of recalibration phases during continuous transfers, an initial secret at installation, and high encryption speed. The document notes that overall security is not unconditional, since it also depends on the encryptor.

UC2 – Enterprise metropolitan area network. Several branches of an enterprise or a public body are connected to data centres, through which branch-to-branch traffic also flows. Each link is protected by a QKD link encryptor that encrypts and authenticates at layer 2, transparently for applications. Two new needs appear: different links carry different traffic volumes and timings, hence *different QoS requirements*, and transfers may be scheduled, periodic or triggered by thresholds. The *QKD Secured Key Server* variant has a central server that distributes keys and certificates to clients across the metropolitan area over channels encrypted with one-time pad and authenticated with Wegman–Carter: an early example of a key service offered by a network.

UC3 – Critical infrastructure control and data acquisition. This case protects the SCADA traffic of a critical infrastructure, with a railway network as the example: here the *authenticity* of commands (switches, signals) matters more than their confidentiality. The nodes of the private WAN sit in stations with a controlled perimeter, while the fibre lies outside it. The document proposes two architectures: single QKD links between nodes, with key buffering and forwarding handled in the nodes, or a *QKD network with integrated key management* and a dedicated interface towards applications. It asks for the highest availability, with precautions for recalibration, failures and denial of service, and for the ability to initialise new redundant links with secrets obtained along an alternative path. It is the first case in which key relay, centralised management and path redundancy appear together.

UC4 – Backbone protection. A fibre in an operator's metropolitan backbone, typically a WDM ring with OADMs or ROADMs, carries a quantum channel alongside the data channels. The keys protect control and management traffic or are offered to customers. There are two actors: the infrastructure provider and the service provider that uses it; backbone nodes are trusted. The requirements are the closest to those of an operator: the QKD system must not reduce the throughput of the WDM system by more than 10%, must be compatible with common

fibres, splices and connectors, its management must *integrate into the management of the WDM system*, and installation and maintenance must be possible for staff without quantum physics expertise.

UC5 – High-security access network. In a passive optical network (PON) for Fiber-to-the-Home, downstream traffic is broadcast to all ONUs, so it must be encrypted to prevent one user from reading another's data. One OLT serves 32 to 128 ONUs and the quantum channel follows the same path as the classical one. The configuration is highly asymmetric: a single, more expensive detector at the OLT serves many sources at the ONUs. Short distances allow high rates, which are needed because the generation capacity is shared among all users. The *QKD Authenticated Sensor Network* variant applies the same scheme to a central station with many sensors and actuators.

UC6 – Long-haul service. Two distant ground stations A and B, for example on opposite shores of an ocean, want to share a key without trusting intermediate ground nodes. A moving node C (satellite, aircraft or high-altitude platform) establishes a secret a with A and then a secret b with B, and sends $a \oplus b$ to B over a classical channel; B recovers a. It is a key relay in which the trusted node is C. Low-orbit satellites fly at about 300–800 km and pass over the stations about once a day; geostationary orbit is not practical. Weather and atmosphere can reduce or cancel the rate, so redundant channels are needed.

Table 1 summarises the six cases.

Table 1: The GS QKD 002 use cases and the elements that anticipate later specifications.

Case	Configuration	Need it anticipates
UC1 Backup	point-to-point link + line encryptor	key–encryptor interface
UC2 Enterprise metro	several links, key server	per-application QoS
UC3 Critical infrastructure	network with key relay and integrated management	KMS, redundancy, centralised control
UC4 Backbone	quantum channel in WDM	coexistence with the optical network, integrated management
UC5 PON access	point-to-multipoint	sharing of key capacity
UC6 Long-haul	relay through a moving node	trusted node, redundancy

2.1.4 What it leaves behind

GS QKD 002 is a first-edition document: the clause on general requirements and the functional characteristics of the individual cases are marked as not yet available. Its value lies in having set the questions. Read in sequence, the use cases show that useful QKD is not a single link but a *network*, which requires: a key management system that stores and relays keys (UC3, UC6); a way to express different service requirements for different applications (UC2); and coexistence with the operator's optical network, with management integrated into the existing one and usable by non-specialist staff (UC4). The first two needs are answered by GS QKD 015, the third, in the form of coordination between the QKD network and the transport network, by GS QKD 018.

2.2 ETSI GS QKD 015: the SDN controller and the SD-QKD network

GS QKD 015 (V1.1.1 in 2021, V2.1.1 in April 2022) [9] defines the control interface between an SDN controller and the nodes of a QKD network, i.e. the *southbound interface* of the QKD control

plane. The second edition, considered here, was aligned with GS QKD 018: the GS QKD 018 YANG modules import those of GS QKD 015 directly. A good understanding of 015 is therefore a prerequisite for understanding 018.

The section proceeds in four steps: why software control is needed; what the components of an SD-QKD network are, one by one; what a YANG model looks like in general; and how 015 uses YANG to describe the node, with the resulting workflows and security considerations.

2.2.1 Why software-defined control

The specification starts from an observation: integrating QKD into a telecommunications network is today a manual operation. Each new QKD system is installed by hand, usually over a dedicated dark fibre between two sites, forming a fixed link. This approach has three limits. It does not scale, because every change requires human intervention. It wastes resources, because the key capacity of a link cannot be moved to where it is needed. And it does not talk to the optical network, which is increasingly disaggregated and software-controlled.

The proposed architecture lets a controller orchestrate QKD resources centrally: optimise the key allocation per link according to demand, automatically create direct or multi-hop links, and coordinate quantum and classical channels even when they share the same fibre. In particular, the abstraction defined by the specification aims to:

- manage QKD resources centrally according to demands, capabilities and network state;
- aggregate the QKD systems of the same perimeter under a single key management, to better detect demands;
- hide the complexity of the low-level parameters of each technology, delegating them to the node;
- optimise the distribution of links according to application QoS and generation statistics;
- coordinate quantum and classical channels, whether or not they share the same physical medium.

These are, in technical form, the needs that emerged from use cases UC2, UC3 and UC4 of GS QKD 002.

2.2.2 The components of an SD-QKD network

It is convenient to build the network bottom-up, adding one abstraction at a time. Fig. 2 shows the result.

1. QKD module, system and link. The basic physical unit is the *QKD module*: the set of hardware and software that runs part of a QKD protocol. Two connected modules form a *QKD system*, and the connection between them (quantum channel plus classical channel) is the *QKD link*. So far we are in the quantum layer of Fig. 1: modules are vendor- and technology-specific (discrete or continuous variables, encoding, wavelength) and do not interoperate.

2. QKD interface. The controller does not need to know how a module works internally, only what it offers to the network. GS QKD 015 therefore introduces an abstraction.

Definition 2.4 (QKD interface)**.** A *QKD interface* (`qkdi`) is the abstract representation of a QKD module within a node: it exposes only the attributes relevant to the network, such as technology, model, wavelength range, maximum supported attenuation and attachment point to an optical switch.

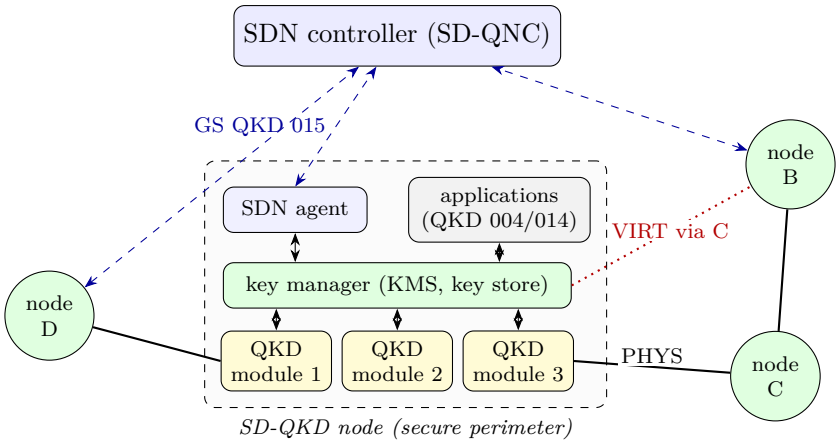


Figure 2: An SD-QKD network: nodes are connected by physical (solid) and virtual (dotted) links, and each is controlled by the controller through its SDN agent (dashed). The node in the foreground shows the internal components: the QKD modules, exposed as interfaces, the key manager, the application interface and the agent. Adapted from [9, Fig. 1].

The analogy is the ports of a switch: a node with three QKD modules appears to the controller as a device with three interfaces. Since modules from different vendors do not interoperate, the model requires the technology and model of each interface to be declared: the controller will pair only compatible interfaces.

3. SD-QKD node. Interfaces located in the same secure site are grouped together.

Definition 2.5 (SD-QKD node)**.** An *SD-QKD node* is the aggregation of one or more QKD modules located in a secure site and managed by a single *SDN agent*, which exposes them to the controller as a single network element with several interfaces.

The specification sets a series of requirements that give the node its shape. An SD-QKD node *shall*: comprise at least one QKD module, exposed as an interface; be located in a secure site (a site may host several nodes); contain a *key management system* that aggregates the material produced by all key associations; offer applications a *single access point* to keys through an API; and connect to at least one SDN controller via standard protocols for discovery, control and telemetry. It *should* also expose to the controller information on applications, interfaces, links and classical-channel requirements (for example acceptable attenuation).

Inside the node four logical components can thus be identified:

- the **QKD modules**, which generate key with remote nodes;
- the **key manager** (KMS), which receives keys from the modules, stores them in one or more logical *key stores*, relays them when needed and registers applications with their QoS;
- the **application interface**, i.e. the key delivery API (GS QKD 004 or 014), reachable at a known IP address and port;
- the **SDN agent**, which translates the state of all of the above into a YANG model, exposes it to the controller, receives configurations and sends notifications.

4. Key association link. Between two nodes the interesting quantity is not the quantum channel itself but the fact that the two nodes share key. GS QKD 015 models exactly this.

Definition 2.6 (Key association link)**.** A *key association link* (`qkd_link`) is a logical key association between two remote SD-QKD nodes. It is of type `PHYS` (direct) if a quantum channel

connects the two nodes, of type `VIRT` (virtual) if the key is obtained by key relay through intermediate trusted nodes.

A physical link corresponds to a QKD link in ITU-T terminology; a virtual link corresponds to a chain of KM links over several QKD links. Two details make the model consistent. First, a virtual link *consumes* key from the physical links it rests on, and is therefore registered as an *internal application* with a reserved bandwidth (`virt_bandwidth`). Second, every intermediate node of a virtual link knows the previous and the next node (`virt_prev_hop`, `virt_next_hop`), so the relay path is configured hop by hop by the controller. For a node to act as a relay it must declare the `key_relay_mode_enable` capability.

5. Applications. From the node's point of view, an application is any entity that requests keys from the key manager.

Definition 2.7 (QKD application)**.** A *QKD application* (`qkd_app`) is a running instance that consumes keys from an SD-QKD node. It is *external* (`CLIENT`) if it is a user, an HSM, an encryptor, a virtual network function or a security protocol; it is *internal* (`INTERNAL`) if it uses keys for the operation of the network itself, such as authentication or relay for a virtual link.

Each application declares a quality of service: maximum and minimum key bandwidth (`max_bandwidth`, `min_bandwidth`, in bit/s), maximum delivery `jitter` in ms, lifetime `ttl` of an unused key in the key store, and whether paths or keys may be shared among several clients. This is the technical answer to the differentiated-QoS need of UC2.

6. SDN controller. At the top sits the controller (in 015 also *SD-QNC*, *Software-Defined QKD Network Controller*), which has the global view of the domain and plays three roles. It acts as the *central registry* of applications, so that an application only needs to know its own key manager and not the network topology. It plans and creates *links*, physical or virtual, according to demand and statistics. And it coordinates its domain with the optical network controller (*SD-ONC*) when a new optical path is needed for a quantum channel.

What does not reach the controller. The information that crosses the control plane is deliberately minimal: identifiers, states, rates, physical parameters. Key material *never leaves the node perimeter*. For this reason, the specification observes, introducing SDN into a trusted-node network adds no new risks beyond those already known, such as denial of service.

2.2.3 Anatomy of a YANG model

Both 015 and 018 are written in YANG [25], a language born in the IETF to describe the configuration and state data of network devices and the operations that can be invoked on them. A YANG model is not executable code: it is a *schema*, like an XSD for XML or a JSON schema, which states which data exist, of which type, how they are nested and who may modify them. The NETCONF and RESTCONF protocols then work on top of that schema. We go through the main constructs using examples taken from the 015 files [26].

The module and its header. The basic unit is the *module*. Its header declares the language version, a globally unique `namespace`, a short `prefix` used by other modules to refer to it, the imported modules and the revisions.

```
module etsi-qkd-sdn-node {
  yang-version "1";
  namespace "urn:etsi:qkd:yang:etsi-qkd-node";
```

```
  prefix "etsi-qkdn";
  import ietf-yang-types   { prefix "yang"; }
  import ietf-inet-types   { prefix "inet"; }
  import etsi-qkd-node-types { prefix "etsi-qkdn-types"; }
  organization "ETSI␣ISG␣QKD";
  revision "2022-01-30" { reference "ETSI␣GS␣QKD␣015␣V2.1.1"; }
  ...
}
```

GS QKD 015 is split into two modules. The first one, `etsi-qkd-node-types`, contains only types; the second one, `etsi-qkd-sdn-node`, contains the actual model. This is a common choice: separating the types lets other modules, such as those of 018, reuse them without importing everything else.

Types: built-in, typedef and identity. YANG offers built-in types (`boolean`, `string`, `uint32`, `decimal64`, `enumeration`…) and common types defined by the IETF, such as `yang:uuid` or `yang:date-and-time`. `typedef` defines derived types with their own name. For extensible symbolic values, `identity` is used: a base identity is declared, then identities that derive from it; a field of type `identityref` accepts any identity derived from the base. This is how 015 defines link types:

```
identity QKD-LINK-TYPES { description "QKD␣key␣association␣link␣types."; }
identity VIRT { base QKD-LINK-TYPES; description "Virtual␣Link."; }
identity PHYS { base QKD-LINK-TYPES; description "Physical␣Link."; }

typedef qkd-link-types {
  type identityref { base QKD-LINK-TYPES; }
}
```

The advantage over an `enumeration` is that another module can add, say, a new link type without modifying the original. Technologies (`CV-QKD`, `DV-QKD`…), node states (`NEW`, `OPERATING`, `DOWN`, `FAILURE`, `OUT`), roles (`TRANSMITTER`, `RECEIVER`, `TRANSCEIVER`) and application types (`CLIENT`, `INTERNAL`) are defined in the same way.

Data nodes: container, list, leaf, leaf-list. Data are organised as a tree with four constructs:

- `leaf`: a single typed value (e.g. `qkdn_id`);
- `leaf-list`: a sequence of values of the same type (e.g. `virt_next_hop`, which allows several successors);
- `container`: a node grouping other nodes, with no value of its own (e.g. `qkdn_capabilities`);
- `list`: a collection of structured entries, each identified by a `key` (e.g. the `qkd_link` list keyed by `qkdl_id`).

Configuration and state. Every node is either *configuration* (`config true`, the default: the client may write it) or *operational state* (`config false`: produced by the device, read-only for the client). In 015, for example, `qkdl_enable` is configurable, while `qkdn_status`, `skr` and `eskr` are state. This distinction mirrors the split between what the controller *decides* and what the node *measures*.

Reuse and extension: grouping, uses, augment. A `grouping` is a reusable block of nodes, copied wherever `uses` appears. The 015 model is built almost entirely this way: `app_`

`basic`, `app_details`, `app_qos`, `qkd_links` and so on are groupings later assembled into the `qkd_node` container. `augment` does the opposite: it adds nodes to a tree defined *in another module*, without touching it. It is the tool that 018 will use to enrich its topology with inventory data.

References and constraints. A `leafref` is a field whose value must match an existing value elsewhere in the tree, like a foreign key in a database. `when` and `must` statements add conditions, for instance that a field exists only if the link is of a certain type. This is exactly how 015 states that the `virt_*` fields apply only to virtual links and the `phys_*` fields only to physical ones.

Operations and notifications: rpc and notification. Besides data, a module can define operations invoked by the client (`rpc`, with `input` and `output`) and messages the server sends spontaneously (`notification`). 015 mostly defines notifications; 018, as we will see, also defines RPCs.

Reading a tree. The `pyang` tool produces a tree view of a module, which we use below. The notation is compact: `rw` marks a configuration node, `ro` a state node, `?` an optional node, `*` a list or leaf-list, a list's key appears in square brackets and `---n` marks a notification.

From data to message. The same schema produces both XML and JSON. In JSON [27] the top level is qualified with the module name, and `identityref` values carry the module that defines the identity. A physical link exposed by a node would look like this:

```
{ "etsi-qkd-sdn-node:qkd_node": {
    "qkdn_id": "0f3c...-a1",
    "qkd_links": { "qkd_link": [ {
        "qkdl_id": "7b2e...-c4",
        "qkdl_type": "etsi-qkd-node-types:PHYS",
        "qkdl_local":  { "qkdn_id": "0f3c...-a1", "qkdi_id": 1 },
        "qkdl_remote": { "qkdn_id": "9d41...-e7", "qkdi_id": 2 },
        "phys_wavelength": 1550,
        "qkdl_performance": { "skr": 20000, "eskr": 12000,
                              "expected_consumption": 8000 } } ] } } }
```

(Values are illustrative.)

2.2.4 The SD-QKD node model

With these tools the 015 model is easy to read. The root container `qkd_node` has four sections, one for each component described above, plus the notifications:

```
module: etsi-qkd-sdn-node
  +--rw qkd_node
     +--rw qkdn_id              yang:uuid
     +--ro qkdn_status?         etsi-qkdn-types:qkdn-status-types
     +--rw qkdn_version?        string
     +--rw qkdn_location_id?    string
     +--rw qkdn_capabilities
     |  +--rw link_stats_support?          boolean
     |  +--rw application_stats_support?   boolean
     |  +--rw key_relay_mode_enable?       boolean
     +--rw qkd_applications
     |  +--rw qkd_app* [app_id]
     |     +--rw app_id, app_type, server_app_id, client_app_id*, ...
     |     +--rw app_qos            (max/min_bandwidth, jitter, ttl, ...)
     |     +--rw backing_qkdl_id*, local_qkdn_id, remote_qkdn_id
```

```
+--rw qkd_interfaces
|  +--rw qkd_interface* [qkdi_id]
|     +--rw qkdi_id, qkdi_model, qkdi_type
|     +--rw qkdi_att_point     (device, port)
|     +--rw qkdi_capabilities  (role, wavelength_range, max_absorption)
|     +--ro qkdi_status
+--rw qkd_links
   +--rw qkd_link* [qkdl_id]
      +--rw qkdl_id, qkdl_status, qkdl_enable, qkdl_type
      +--rw qkdl_local / qkdl_remote   (qkdn_id, qkdi_id)
      +--rw qkdl_applications*
      +--rw virt_prev_hop, virt_next_hop*, virt_bandwidth   (VIRT)
      +--rw phys_channel_att, phys_wavelength, phys_qkd_role (PHYS)
      +--rw qkdl_performance   (expected_consumption, skr, eskr,
                                phys_perf* [perf_type]: QBER, SNR)
```

Node identity. `qkdn_id` is a UUID unique in the network; `qkdn_status`, `qkdn_version` and `qkdn_location_id` serve inventory and diagnostics.

Capabilities. The three capabilities tell the controller what the node is willing to do: expose link statistics (SKR, consumption, status, QBER), expose application statistics, and act as a relay. A node with `key_relay_mode_enable = false` will never be chosen as an intermediate node of a virtual link.

Applications. Each `qkd_app` binds an application to the key service: `server_app_id` identifies the entity that initiated the request, `client_app_id` the entities allowed to receive the same key, `backing_qkdl_id` the link that supplies the keys, `local_qkdn_id` and `remote_qkdn_id` the two nodes involved. Until the remote node is known, the local node cannot serve keys. Statistics (`app_statistics`) report the bits consumed per time interval.

Interfaces. Each `qkd_interface` describes a module: model, technology, status, attachment point to an optical device (`qkdi_att_point`) and physical capabilities. This is the information the controller needs to pair two compatible interfaces and request an optical path between their ports.

Links and performance. Each `qkd_link` reports its endpoints, type, status, the applications it serves and the type-specific parameters. The `qkdl_performance` block holds the three quantities that drive all planning:

- `skr`: secret key generation rate, in bit/s;
- `expected_consumption`: sum of the bandwidths of all applications on the link;
- `eskr`: *effective* rate, i.e. what is available after internal consumption.

In compact form, if $\mathcal{A}_\ell$ is the set of applications on link ℓ, each with bandwidth b_a,

$$\text{expected_consumption}_\ell = \sum_{a \in \mathcal{A}_\ell} b_a, \qquad \text{ESKR}_\ell \approx \text{SKR}_\ell - \text{expected_consumption}_\ell. \tag{2}$$

The second relation is an intuitive reading of the fields, not a formula of the standard. Physical links also carry metrics such as QBER and SNR.

2.2.5 Notifications

Polling the nodes continuously would be inefficient. GS QKD 015 therefore defines YANG notifications from node to controller, grouped by object:

- **applications**: `sdqkdn_application_new` (a new application requests keys), `sdqkdn_application_qos_update`, `sdqkdn_application_disconnected`;
- **interfaces**: `sdqkdn_interface_new`, `sdqkdn_interface_down` (with a *reason* field), `sdqkdn_interface_out`;
- **links**: `sdqkdn_link_down`, `sdqkdn_link_perf_update` (significant performance change), `sdqkdn_link_overloaded` (the link cannot cope with demand).

In addition there is a generic structure for events and alarms, designed to accommodate new event types without changing the model. Moreover, YANG-based protocols allow subscribing to any datum in the datastore, so that every change can be reported in real time.

2.2.6 Workflows

Clause 5 describes three fundamental scenarios that show the components at work.

Application registration. This is the most instructive case, because it explains why the controller acts as a central registry (Fig. 3). An application knows only the address of its own key manager; it does not know the node of the application it wants to talk to.

1. The application in security domain A connects to its node through the key delivery API, sending its own ID, the ID of the peer application and the requested QoS.
2. Node A does not know where the peer application is and forwards the request to the controller, adding its own ID.
3. If the peer application is not yet registered, the controller replies with a plain acknowledgement.
4. In domain B the peer application connects to its node, which in turn informs the controller.
5. The controller matches the pair, generates a unique service identifier (`app_id`) and sends both nodes all the information they need: application and node IDs, `app_id` and QoS.
6. The nodes pass the `app_id` to the applications and reserve keys for them; from then on the applications request keys using that identifier.

Physical link creation. The controller receives a trigger, from a demand detected on the nodes or from its own NBI. It looks for two free, compatible interfaces at the two sites, asks the optical network controller (SD-ONC) for an optical path (*lightpath*) with the constraints of the quantum channel and, once confirmed, configures the two interfaces on their nodes. If the two controllers are merged into one, the operation is internal. This workflow is the core of the QKD–optical network coexistence demanded by UC4.

Virtual link creation. If there are no compatible interfaces or no suitable lightpath, the controller falls back to a virtual link. It computes a path over the existing QKD network, for example on a weighted graph built from key availability, generation rates and consumption; it configures each intermediate node in relay mode, indicating previous and next hop; finally it configures the endpoints with the bandwidth to provide. The specification does not prescribe

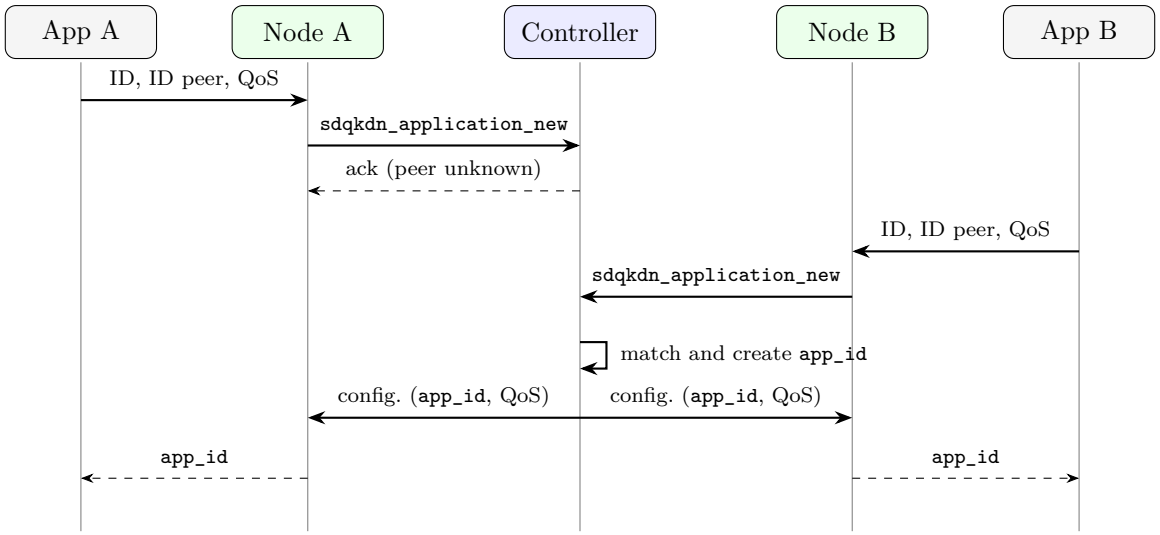


Figure 3: Application registration in an SD-QKD network. Applications know only their own key manager; the controller matches the two requests and creates the service identifier. Adapted from [9, Fig. 2].

the algorithm. A natural choice, consistent with (2), is to weight each link with the inverse of its available key and discard links that cannot sustain the requested bandwidth r:

$$w_\ell = \frac{1}{\mathrm{ESKR}_\ell}, \qquad p^\star = \arg \min_{p \,:\, \min_{\ell \in p} \mathrm{ESKR}_\ell \geq r} \sum_{\ell \in p} w_\ell. \tag{3}$$

Equation (3) is our example, not part of the standard. The specification warns that, if poorly managed, virtual links can congest the underlying physical links: this is precisely why a controller with a global view of consumption and availability is needed.

2.2.7 Protocols and security

GS QKD 015 does not mandate a protocol, but points to the two most common YANG-based ones: NETCONF [28], typically between device and controller, transactional and carried over SSH or TLS with XML encoding; and RESTCONF [29], typically on the controllers' NBI, based on HTTP with CRUD operations and JSON or XML encoding.

On security (clause 6) the specification distinguishes two cases when a new system is installed. If the system joins a secure site already in the network, the channel to the controller can be protected with QKD keys produced by existing links. If a new secure site is added, an initial scheme (a trusted identity or a pre-shared secret, used only during installation) is needed to introduce the new SDN agent to the controller; after the first authentication, QKD keys take over. It is the same key-growing principle seen at the beginning.

2.2.8 From a single domain to orchestration

GS QKD 015 solves the control of *one* QKD domain by *one* controller. Three problems already hinted at by GS QKD 002 are left out. First, an operator may have several QKD domains, possibly from different vendors, each with its own controller. Second, the applications that consume key may live in another network, for example in the nodes of the optical transport network, managed by a different controller. Third, the operator wants to manage QKD with the same orchestration tools it uses for the rest of the network. A higher layer is therefore needed, together with a standard interface to reach it: this is the job of GS QKD 018.

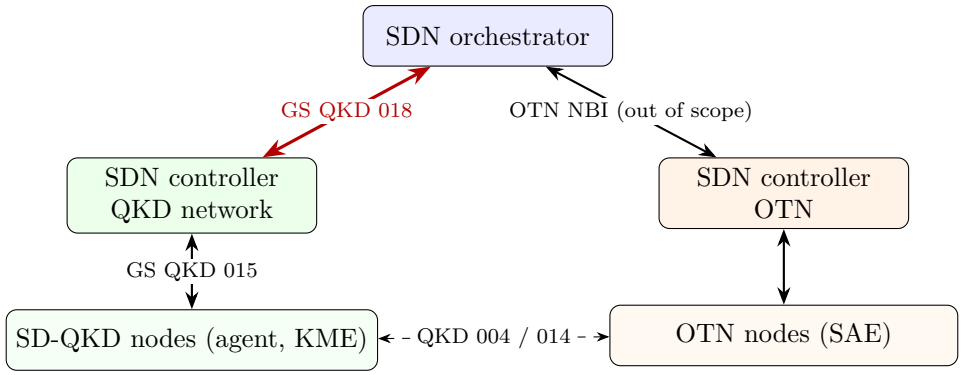


Figure 4: GS QKD 018 reference scenario (adapted from [10, Fig. 1]). The standardised interface is in red; the dashed line is key delivery from KMEs to SAEs, which happens inside the same security perimeter.

3 ETSI GS QKD 018

GS QKD 018 (V1.1.1, April 2022) [10] defines the *orchestration interface* between an SDN orchestrator and the SDN controller of a QKD network. The controller acts as *server*, the orchestrator as *client*; the information model is expressed in YANG and is vendor-independent. In SDN terms it is the *northbound interface* of the QKD controller, i.e. the exact complement of GS QKD 015. The overall hierarchy becomes:

$$\text{orchestrator} \xrightarrow{\text{QKD 018}} \text{controller} \xrightarrow{\text{QKD 015}} \text{SDN agent / SD-QKD node} \xrightarrow{\text{QKD 004/014}} \text{SAE}.$$

Definition 3.1 (SDN orchestration)**.** *SDN orchestration* is the continuous process by which available resources are automatically coordinated, according to optimisation criteria, to establish and release end-to-end services that span several network domains, each governed by its own SDN controller [10, §4.2].

Definition 3.2 (Service link)**.** A *service link* is a logical key association link between two QKD nodes, realised by one or more physical QKD links (single-hop or multi-hop). At the orchestration level it corresponds to the virtual link of GS QKD 015.

3.1 GS QKD 018 use cases

The use case behind the standard (§4.1) takes up almost literally the backbone problem of GS QKD 002. An operator introducing QKD can either let quantum and classical channels coexist in the same fibre or keep them separate. Coexistence is hard for two reasons: the optical powers of classical channels are many orders of magnitude higher and disturb the quantum channel, and the reachable distances of the two kinds of channel are very different. In a fibre-rich environment the operator may therefore choose a QKD network separate from the optical transport network (OTN), gaining performance, isolation and separation of responsibilities. Following the SDN paradigm, the two networks are governed by different controllers.

This creates a coordination problem: the SAEs that need to encrypt traffic sit in OTN nodes, keys are generated in QKD nodes, and each controller knows only its own network. The orchestrator solves three issues:

1. **Address matching**: it pairs the address of the OTN node that must receive keys with that of the QKD node that supplies them; the two are inside the same security perimeter (Fig. 4);

2. **end-to-end provisioning**: it translates a customer request into the configuration of the individual domains, chooses the domains to traverse and coordinates the paths;
3. **management**: it collects the topology, key-resource inventory and FCAPS information of each domain.

Generalising, the same scheme makes it possible to orchestrate *several QKD domains from different vendors*, each behind its own controller, as well as QKD and classical domains together. The standard also points out a practical advantage: an operator that already has an orchestrator for its optical networks can integrate the QKD network without introducing separate management tools, which is exactly the integrated management requirement of UC4.

The functions required of the orchestrator (§4.2) are:

- translating customer service requests into domain configuration and SAE allocation;
- requesting establishment and release of services, including inter-domain connections;
- computing multi-domain paths, with inter-domain connections and endpoints;
- changing paths under operator constraints;
- discovering the nodes and links of each controller and keeping an abstract view of them;
- monitoring the key resources available on nodes and links;
- monitoring FCAPS management and receiving its notifications.

3.2 The information model

Clause 5 translates these functions into five information blocks.

Topology discovery (§5.1). The controller exposes SD-QKD nodes, interfaces and physical links, and later service links. The orchestrator does not need the full physical composition, but an *abstract* view of each domain and of the connectivity between domains. Discovery can be *proactive* (the orchestrator queries the controller before every new path computation) or *reactive* (the controller notifies every change, e.g. a node added or removed).

Network status and inventory (§5.2). For each node and physical link the controller exposes status, physical performance and above all key resources: the generation rate and the rate available after internal consumption. Service links add bandwidth, available key and the restoration policy in case of failure.

End-to-end service status (§5.3). The orchestrator can query the active external applications, the local and remote nodes and the service links that serve them, with their status and key rate.

Provisioning with path computation (§5.4). The orchestrator asks the controller for paths that satisfy given constraints; the controller returns a list of candidates, possibly several times with different constraints, until the orchestrator picks one to deploy. In case of failure the orchestrator can request the path to be replaced.

Notifications (§5.5). The controller reports events, alarms and threshold crossings. Performance thresholds (QBER, SKR, ESKR) are configurable by the orchestrator. Compared with 015, notifications are added for node status (`sdqkdn_new`, `_operating`, `_down`, `_failure`, `_out`),

for deployed applications (`sdqkdn_application_deployed`) and for service links (`sdqkdn_svc_link_down`, `_perf_update`, `_overloaded`).

Equation (2) extends naturally to the orchestration level. Since in a multi-hop path p every link relays the same key, a new service with demand r is sustainable only if the bottleneck can carry it:

$$p \text{ admissible} \iff \min_{\ell \in p} \mathrm{ESKR}_\ell \geq r. \tag{4}$$

This too is our own formalisation, consistent with the `skr`, `eskr` and `expected_consumption` fields of the YANG models; the standard does not prescribe any path-computation algorithm.

3.3 The YANG modules

Annex A (normative) turns the model into three YANG 1.1 modules, published on ETSI Forge [18]. With the constructs seen in §2.2.3 their structure is quick to read. The modules import `etsi-qkd-sdn-node` and `etsi-qkd-node-types` from GS QKD 015 to reuse their types (identifiers, link types, application QoS): 018 does not reinvent the node, it provides an abstract view of it. The three modules form a chain: *topology* (the skeleton) → *inventory* (what is there and in which state) → *connectivity* (what is done with it).

`etsi-qkd-sdn-topology` (prefix `qkdnw-tplg`). It describes only the structure, i.e. who is connected to whom:

```
qkd_network
 +-- sdqkd_nodes
 |    +-- qkdn* [qkdn_id]                            SD-QKD nodes
 |         +-- qkd_interfaces/qkdi* [qkdi_id]       QKD modules
 +-- qkd_links
      +-- qkd_phys_links/phys_link* [phys_link_id]  physical links
      |    +-- link_type
      |    +-- phys_link_local_qkdn   (qkdn_id_ref, qkdi_id_ref)
      |    +-- phys_link_remote_qkdn  (qkdn_id_ref, qkdi_id_ref)
      +-- qkd_svc_links/svc_link* [svc_link_id]     service links
           +-- ... local/remote endpoints as above
```

Compared with 015 the point of view changes: there is no longer a tree per node, but a single tree for the whole network of the domain. Abstractly it is a graph $G = (V, E)$ with V the set of `qkdn` nodes and $E = E_{\text{phys}} \cup E_{\text{svc}}$, where `phys_link`s represent physical adjacencies and `svc_link`s logical key relations, possibly multi-hop. Endpoints are `leafref`s, i.e. validated references to existing nodes and interfaces. Given the graph G_i of each domain i, the orchestrator composes a multi-domain view

$$G = \Big(\bigcup_i V_i,\ \bigcup_i E_i \cup E_{\text{inter}} \Big), \tag{5}$$

where E_{inter} are the adjacencies between domains, typically realised by shared border nodes.

`etsi-qkd-sdn-inventory` (prefix `qkdnw-invt`). It does not create a new tree but `augment`s the topology tree, adding leaves to elements already defined (Table 2). It is a textbook use of `augment`: those interested only in the structure implement the topology, those who also want the state add the inventory without duplicating anything.

The module also defines the notifications that implement the reactive part of §5.5: `sdqkdn_new`, `sdqkdn_interface_new`, `sdqkdn_phys_link_new`, `sdqkdn_svc_link_new`, the `_perf_update` and `_overloaded` variants and the generic `alarm` and `event`. The `eskr` field of the links is what tells the orchestrator whether a path can carry a new service, according to (4).

Table 2: Elements added by the inventory module to the topology tree.

Element	Added leaves
`qkdn`	version, capabilities, location, KME IP address and port (where SAEs fetch keys via QKD 014), status
`qkdi`	supported role, model, type, attachment point, status
`phys_link`	`enable` (configurable), `wavelength`, `performance` (`skr`, `eskr`, `expected_consumption`, QBER, SNR), status
`svc_link`	`bandwidth`, `performance`, `path_restoration` (`AUTOMATIC`/`MANUAL`, `hold_off_time` in ms), status

`etsi-qkd-sdn-connectivity` (prefix `qkdnw-conn`). This is the operational module: it manages applications and the paths that serve them.

```
connectivity_service/qkd_service/qkd_app* [app_id]
 +-- (application data from QKD 015: status, type, QoS, statistics)
 +-- app_mapped_svc_link
      +-- svc_link_and_qkdn_ids   service link and endpoints
      +-- path_constraints        include_nodes / exclude_nodes
      +-- candidate_paths         num_of_candidate_paths + list of paths
      +-- deploy_path             chosen path_id
```

Here the `rpc`s appear, mirroring the provisioning workflow step by step:

`app-registration`
: registers the application in the domain;

`create-service-link-id-for-app`
: creates the service link between local and remote node;

`create-candidate-paths-per-service-link-for-app`
: the controller computes candidate paths honouring QoS and the `include_nodes`/`exclude_nodes` constraints;

`update-candidate-paths-per-service-link-for-app`
: recomputation with different constraints;

`deploy-a-candidate-path-per-service-link-for-app`
: the orchestrator picks a `path_id` and the controller activates it;

`delete-deployed-path-per-service-link-for-app`
: releases the path.

The module's notifications are `sdqkdn_application_new`, `sdqkdn_application_deployed`, `sdqkdn_application_qos_update` and `sdqkdn_application_disconnected`.

The conceptual point is the **division of roles**: the controller knows its own domain and *proposes* paths, the orchestrator has the overall view and *decides*. Table 3 sums up.

Table 3: The three YANG modules of GS QKD 018.

Module	Question	Clause	Data
topology	What does the network look like?	§5.1	structure (graph)
inventory	How many resources, in which state?	§5.2, §5.5	status, performance, configuration, notifications
connectivity	How do I serve an application end to end?	§5.3, §5.4, §6.2	applications, path computation and deployment RPCs

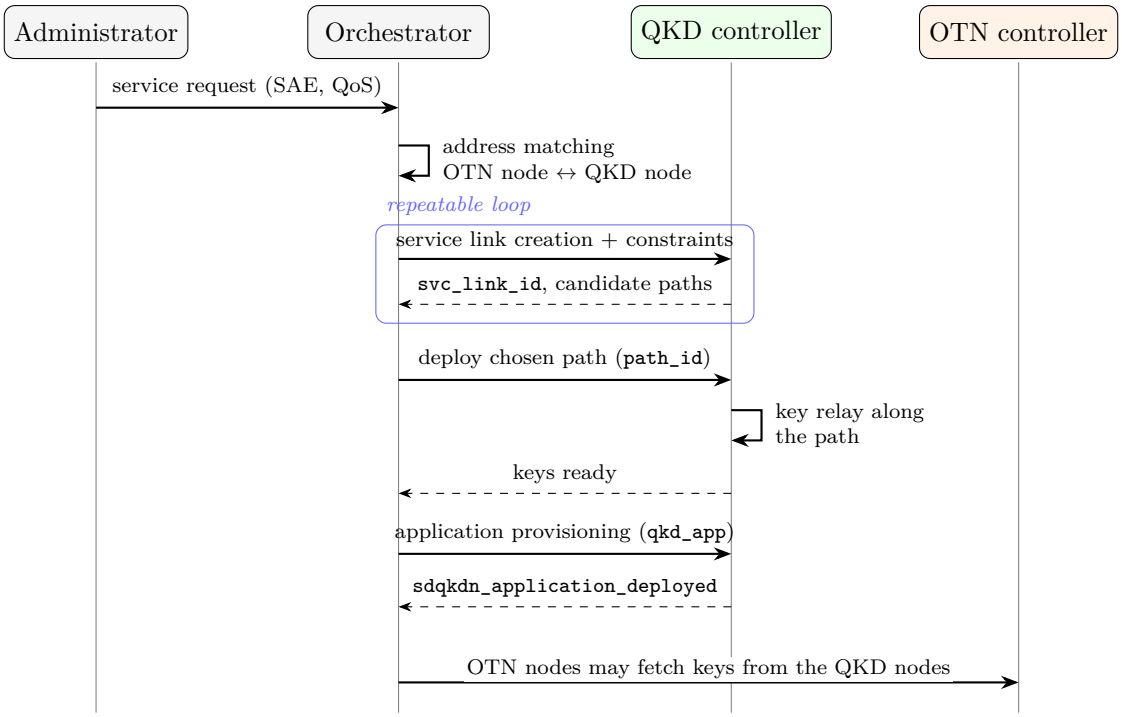


Figure 5: End-to-end provisioning of a QKD service across a QKD network and an OTN (simplified from [10, Fig. 2]). The interaction with the OTN controller is out of the scope of the specification.

3.4 Multi-domain provisioning workflow

Clause 6 ties the information to the service life cycle. When the QKD network is introduced, the orchestrator acquires its topology (§5.1); during operation it receives status and inventory, on demand or periodically (§5.2); after a service is activated it monitors its status (§5.3); at any time it receives notifications (§5.5).

§6.2 describes the complete case (Fig. 5): an orchestrator coordinates the QKD controller and the OTN controller, which do not talk to each other directly.

The steps are:

1. the administrator requests a service from the orchestrator, specifying the SAEs in the OTN nodes and their QoS;
2. the orchestrator identifies the controllers involved and performs address matching; it already knows the addresses of the QKD nodes thanks to topology and inventory;
3. it asks the QKD controller to create a service link and compute paths, with the desired constraints;
4. the controller assigns an `svc_link_id` and returns the candidate paths, each with a priority; this step may be repeated with different constraints;
5. the orchestrator picks a path and requests its deployment; the nodes generate and relay keys along the path until both ends are ready;
6. once confirmed, the orchestrator requests provisioning of the application on the service link and tells the OTN controller that the OTN nodes may start fetching keys from the designated QKD nodes.

Note the parallel with the virtual-link workflow of 015: there the controller computed and configured the path on its own, here it *proposes* it and waits for the orchestrator's choice. With

several QKD domains the orchestrator repeats steps 3–5 for each domain along the chosen inter-domain path, joining the segments through border nodes: this is what Sim et al. do (§5.1).

3.5 Protocols and security

Transport (§8). Like 015, the standard separates the data model (YANG 1.1) from the transport protocol, which can be NETCONF [28] or RESTCONF [29]. With RESTCONF every resource is a URI, reads use `GET` and configuration uses `POST`, `PUT`, `PATCH` and `DELETE`, with XML or JSON encoding; notifications are received by subscribing to the corresponding streams. The standard notes that the stateless REST paradigm suits this kind of API. As an example, reading the topology and invoking an RPC would look like:

```
GET  /restconf/data/etsi-qkd-sdn-topology:qkd_network
POST /restconf/operations/etsi-qkd-sdn-connectivity:create-service-link-id-for-app
```

Security (§7). The controller has a central view of the network and passes a subset of it to the orchestrator; protection measures for the interface (authentication, channel encryption, e.g. TLS under RESTCONF) are left to implementations. The fundamental principle, inherited from GS QKD 015, is that **QKD-derived keys never pass through either the controller or the orchestrator**: the YANG modules contain only key rates and the KME address. A compromised orchestrator can therefore cause service disruption (wrong paths, exhausted resources, denial of service) but cannot read keys. The standard recommends careful design, installation and personalisation procedures to mitigate malicious requests and attacks on control channels.

4 The analogy with ACTN (RFC 8453)

GS QKD 018 does not invent a new model: it applies to QKD a pattern that is already standard in transport networks, the IETF ACTN framework (*Abstraction and Control of TE Networks*) [30]. ACTN has three levels: the *Customer Network Controller* (CNC), the *Multi-Domain Service Coordinator* (MDSC) and the per-domain *Provisioning Network Controllers* (PNC), connected by the CMI (CNC–MDSC) and MPI (MDSC–PNC) interfaces. The mapping to GS QKD 018 is direct (Table 4).

Table 4: Mapping between ACTN and the ETSI QKD stack.

ACTN	ETSI QKD	Remark
CNC	administrator / customer portal	originates the service request
MDSC	SDN orchestrator	multi-domain view, path decision
PNC	SDN controller of the QKD domain	computes candidates, activates paths
MPI	GS QKD 018 interface	abstract topology, provisioning, notifications
SBI	GS QKD 015 interface	control of individual nodes
abstract TE topology	`etsi-qkd-sdn-topology`	vendor-neutral view
TE bandwidth	SKR / ESKR	the resource to allocate is key

The essential difference lies in the resource. In ACTN the bandwidth of a link is fixed by the channel capacity; in QKD the SKR fluctuates with physical conditions and key accumulates in finite buffers from which it is consumed. This is why in 018 notifications such as `_perf_update` and `_overloaded` and the `path_restoration` policy play a more central role than in classical transport networks.

5 Real deployments

5.1 SK Telecom: multi-vendor orchestration on KOREN

Sim et al. [19] describe the first documented commercial deployment of a GS QKD 018-compliant orchestrator, on the testbed of the Korean research network KOREN (2020–2022). Two QKD operators run two networks with devices and KMSs from different vendors, each with its own controller (QKDNC); an orchestrator (QKDNO) coordinates them. Links use decoy-state BB84 over spans between 39 and 100 km. The two networks meet at *border nodes* (e.g. Pangyo and Daejeon) that contain no QKD modules but two interconnected KMSs, one per operator.

When two SAEs belong to different networks, the orchestrator computes the end-to-end path with a *path computation engine*, choosing *in advance* the border node to traverse; each controller then creates its own service-link segment. To bind the segments the authors extend the model: they introduce a *multi-segment* application type (`MS`), with an identifier `ms_app_id` shared across domains and `server_ms_info` and `client_ms_info` describing the external KMSs, plus an `app-registration-ms` RPC. Key transfer at the border node uses the KMS–KMS interface of GS QKD 020 [11]. The case shows both the soundness of the 018 scheme and one of its limits: the application types inherited from 015 (`CLIENT` and `INTERNAL`) are not enough for multi-domain, and extensions are needed. It is also a good illustration of the usefulness of the YANG `identity` construct discussed in §2.2.3: a new type can be added without rewriting the original model.

5.2 Politecnico di Milano: hierarchical control with ONOS

Vicario's thesis [20] builds an end-to-end prototype of hierarchical control for metropolitan networks: the domain controllers are based on ONOS and manage the nodes through the GS QKD 015 model, while the upward interface is GS QKD 018, exposed via RESTCONF to a Python orchestrator that aggregates the topology of several domains. It is a useful implementation because it shows the whole 015 → 018 stack in open software, with the same YANG modules described in §3.3.

5.3 MadQCI: the Madrid network

MadQCI [21] is a real QKD network in Madrid with dozens of modules from different vendors over more than 130 km, integrated with the production optical network. Control relies on an SD-QKD stack based on GS QKD 015, which is confirmed as the most widely adopted standard. GS QKD 018 is mentioned as a tool for inter-domain orchestration and optical–quantum coordination, but the authors describe it as work still in progress. This is an honest picture of the state of the art: domain control is mature, orchestration much less so.

6 Limits and outlook

- **Adoption.** 015 is used in real networks, 018 almost only in testbeds and prototypes [21].
- **Incomplete multi-domain support.** The normative scenario is QKD+OTN; the QKD+QKD case requires extensions like those of Sim et al. and coordination with GS QKD 020 for key hand-over between KMSs.
- **Metrics.** ESKR is an instantaneous indicator and accounts neither for key already accumulated in buffers nor for its dynamics. Laschet et al. [23] propose richer indicators for path selection.

- **Topology confidentiality.** Exposing the internal topology to the orchestrator is acceptable within one operator, less so between different operators; inter-domain routing schemes that preserve confidentiality are needed [23, 24].
- **Interface security.** The standard does not specify how the orchestrator–controller channel is protected, leaving it to implementations.
- **Algorithms.** Path computation is not standardised: this helps innovation [5, 22] but makes behaviour less predictable in multi-vendor scenarios.

7 Conclusions

Read in sequence, ETSI's work on QKD control follows a clear logic. GS QKD 002 showed that useful QKD is a network, with key management, differentiated service requirements and integration into the operator's optical network. GS QKD 015 answered with the SD-QKD node abstraction: modules as interfaces, a single key manager, physical and virtual links, applications with QoS, and a controller that registers, plans and configures, all described in YANG. GS QKD 018 added the orchestration layer: each domain becomes an abstract graph, key resources are monitored through the inventory and provisioning follows a scheme in which the controller proposes and the orchestrator decides. The design mirrors ACTN, reuses YANG, NETCONF and RESTCONF and keeps the essential security principle: keys never leave the nodes. The KOREN, Politecnico di Milano and MadQCI experiences show that the scheme works but that QKD orchestration is still young. The next step is to truly extend it to multi-operator settings, with better metrics and confidentiality guarantees between domains.

References

[1] Stefano Pirandola et al. "Advances in quantum cryptography". In: *Advances in Optics and Photonics* 12.4 (2020).

[2] Miralem Mehic et al. "Quantum Key Distribution: A Networking Perspective". In: *ACM Computing Surveys* 53.5 (2020).

[3] Yuan Cao, Yongli Zhao, Qin Wang, Jie Zhang, Soon Xin Ng, and Lajos Hanzo. "The Evolution of Quantum Key Distribution Networks: On the Road to the Qinternet". In: *IEEE Communications Surveys & Tutorials* (2022).

[4] GÉANT GN5-1. *White Paper: QKD Concepts and Considerations*. Tech. rep. GÉANT, 2024. URL: https://resources.geant.org/wp-content/uploads/2024/02/GN5-1_White-Paper_QKD-Concepts-and-Considerations.pdf.

[5] *QKD Routing Protocol in Quantum Networks: Overview and Challenges*. arXiv:2407.13156. 2024.

[6] ETSI ISG QKD. *Quantum Key Distribution (QKD); Use Cases*. Group Specification GS QKD 002 V1.1.1. ETSI, June 2010. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/002/01.01.01_60/gs_QKD002v010101p.pdf.

[7] ETSI ISG QKD. *Quantum Key Distribution (QKD); Application Interface*. Group Specification GS QKD 004 V2.1.1. ETSI, 2020. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/004/02.01.01_60/gs_QKD004v020101p.pdf.

[8] ETSI ISG QKD. *Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API*. Group Specification GS QKD 014 V1.1.1. ETSI, 2019.

[9] ETSI ISG QKD. *Quantum Key Distribution (QKD); Control Interface for Software Defined Networks*. Group Specification GS QKD 015 V2.1.1. ETSI, Apr. 2022. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/015/02.01.01_60/gs_QKD015v020101p.pdf.

[10] ETSI ISG QKD. *Quantum Key Distribution (QKD); Orchestration Interface for Software Defined Networks*. Group Specification GS QKD 018 V1.1.1. ETSI, Apr. 2022. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/018/01.01.01_60/gs_qkd018v010101p.pdf.

[11] ETSI ISG QKD. *Quantum Key Distribution (QKD); Interoperable Key Management System*. Group Specification GS QKD 020. ETSI. URL: https://forge.etsi.org/rep/qkd/gs020-interop-kms.

[12] ITU-T. *Overview on networks supporting quantum key distribution*. Recommendation Y.3800. International Telecommunication Union, 2019.

[13] ITU-T. *Quantum key distribution networks – Control and management*. Recommendation Y.3804. International Telecommunication Union, 2025. URL: https://www.itu.int/dms_pubrec/itu-t/rec/y/T-REC-Y.3804-202504-I!!SUM-HTM-E.htm.

[14] ITU-T. *Quantum key distribution networks – Software-defined networking control*. Recommendation Y.3805. International Telecommunication Union, 2021.

[15] Diego Kreutz, Fernando M. V. Ramos, Paulo Veríssimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. "Software-Defined Networking: A Comprehensive Survey". In: *Proceedings of the IEEE* 103.1 (2015).

[16] Alejandro Aguado et al. "First experimental demonstration of secure NFV orchestration over an SDN-controlled optical network with time-shared QKD resources". In: *European Conference on Optical Communication (ECOC)*. arXiv:1604.05861. 2016.

[17] Alejandro Aguado, Victor López, Juan Pedro Brito, Antonio Pastor, Diego R. López, and Vicente Martín. "Enabling Quantum Key Distribution Networks via Software-Defined Networking". In: *International Conference on Optical Network Design and Modeling (ONDM)*. 2020. DOI: 10.23919/ONDM48393.2020.9133024.

[18] ETSI ISG QKD. *gs018-orch-int-sdn: YANG modules etsi-qkd-sdn-topology, etsi-qkd-sdn-inventory, etsi-qkd-sdn-connectivity (rev. 2022-02-22)*. ETSI Forge. 2022. URL: https://forge.etsi.org/rep/qkd/gs018-orch-int-sdn.

[19] Dong-Hi Sim, Jongyoon Shin, and Min Hyung Kim. "Software-Defined Networking Orchestration for Interoperable Key Management of Quantum Key Distribution Networks". In: *Entropy* 25.6 (2023), p. 943. DOI: 10.3390/e25060943.

[20] E. Vicario. "Hierarchical SDN control for advanced quantum-secured metropolitan networks". MA thesis. Politecnico di Milano, 2025. URL: https://www.politesi.polimi.it/handle/10589/246504.

[21] A. Sebastián-Lombraña et al. *A blueprint for large-scale quantum-network deployments*. arXiv:2409.01069. 2024.

[22] R. B. Méndez et al. "Switching Coordinator: An SDN Application for Flexible QKD Networks". In: *Entropy* 28 (2026). arXiv:2603.13812, p. 219.

[23] S. Laschet et al. "SDN Key Relay for Large-Scale QKD Networks". In: *IEEE OCNC*. arXiv:2608.17539. 2026.

[24] *Interconnecting Regional QKD Networks: Hybrid Key Delivery Across Quantum Domains*. arXiv:2604.20376. 2026.

[25] Martin Bjorklund. *The YANG 1.1 Data Modeling Language*. RFC 7950. IETF, 2016. URL: https://www.rfc-editor.org/rfc/rfc7950.

[26] ETSI ISG QKD. *gs015-ctrl-int-sdn: YANG modules etsi-qkd-sdn-node and etsi-qkd-node-types*. ETSI Forge. 2022. URL: https://forge.etsi.org/rep/qkd/gs015-ctrl-int-sdn.

[27] Ladislav Lhotka. *JSON Encoding of Data Modeled with YANG*. RFC 7951. IETF, 2016. URL: https://www.rfc-editor.org/rfc/rfc7951.

[28] Rob Enns, Martin Bjorklund, Jürgen Schoenwaelder, and Andy Bierman. *Network Configuration Protocol (NETCONF)*. RFC 6241. IETF, 2011. URL: https://www.rfc-editor.org/rfc/rfc6241.

[29] Andy Bierman, Martin Bjorklund, and Kent Watsen. *RESTCONF Protocol*. RFC 8040. IETF, 2017. URL: https://www.rfc-editor.org/rfc/rfc8040.

[30] Daniele Ceccarelli and Young Lee. *Framework for Abstraction and Control of TE Networks (ACTN)*. RFC 8453. IETF, 2018. URL: https://www.rfc-editor.org/rfc/rfc8453.